



SolarShare — Peer-to-Peer Solar Energy Trading & Carbon Credit Platform

Comprehensive Project Report & Technical Blueprint (10+ Pages Specification)

TABLE OF CONTENTS

1. Simple Abstraction & Executive Summary
 2. Folder & Project Structure
 3. REST API Specifications (All 11 Modules)
 4. MongoDB Database Schemas & Modeling
 5. Technologies & Express Libraries Breakdown
 6. Real-Time Use Cases & Competitive Edge (Why SolarShare is Better)
 7. How to Work on the Project (Setup & Execution Guide)
-

1. Simple Abstraction & Executive Summary

1.1 What is SolarShare?

SolarShare is an automated, decentralized Peer-to-Peer (P2P) solar energy trading and green carbon credit platform. It directly connects rooftop solar panel owners (**Prosumers**) who generate excess electricity with nearby buyers (**Consumers**) who need clean energy at affordable prices.

1.2 The Problem in Existing Grids

In traditional DISCOM (Distribution Company) net-metering systems:

1. **Low Returns for Solar Owners:** DISCOMs buy surplus rooftop solar energy at a low feed-in tariff (typically ₹2.00–₹3.00/kWh).
2. **High Bills for Energy Consumers:** Consumers pay high slab and peak rates (₹7.50–₹10.00/kWh) for grid power.

3. **No Origin Tracking:** Consumers cannot verify if the electricity they purchase is generated from clean renewable energy.
4. **Delayed Settlements:** DISCOM net-metering credit adjustments take 1 to 2 billing cycles.

1.3 The SolarShare Solution & Innovation

SolarShare bypasses middleman monopolies by providing a direct P2P digital marketplace:

- **Prosumers sell surplus energy at market-driven prices** (e.g. ₹5.00/kWh), earning up to **60% higher revenue**.
- **Consumers buy local clean energy at discounted rates** (e.g. ₹5.50/kWh), saving up to **30% on electricity bills**.
- **Real-Time IoT Smart Meter Simulation:** Streams live generation (kW), consumption (kW), and grid voltage (V) over WebSockets.
- **Escrow-Backed Digital Wallet:** Instant settlement upon trade execution with zero counterparty risk.
- **Tokenized Carbon Credit Certificates:** Mints verified Renewable Energy Certificates (RECs) based on actual $\text{\text{CO}}_2$ emissions avoided ($\$0.85 \text{ kg } \text{CO}_2 / \text{kWh}$).

2. Folder & Project Structure

SolarShare uses a modular, decoupled Client-Server architecture.

```

SolarShare/
├── client/                                     # React.js 18 + Vite Single Page Application
│   ├── public/                               # Static icons & images
│   └── src/
│       ├── api/                             # Axios REST API config
│       │   └── axios.js                     # Base URL & Authorization Bearer header interceptor
│       ├── components/                      # Global Reusable UI Components
│       │   ├── Footer.jsx                   # Site footer
│       │   ├── Navbar.jsx                   # Header navigation & role-based route links
│       │   ├── ProtectedRoute.jsx           # Route guard for JWT auth & role authorization
│       │   └── DashboardLayout.jsx          # Sidebar navigation layout for dashboards
│       ├── context/                         # Application State Management
│       │   └── AuthContext.jsx              # Auth provider managing JWT session & user profiles
│       ├── pages/                           # User Dashboard Screens
│       │   ├── Landing.jsx                  # Landing homepage
│       │   └── admin/                       # Admin Command Center Views
│       │       ├── AdminDashboard.jsx        # Grid health summary & revenue metrics
│       │       ├── AdminUsers.jsx           # User management & account suspension
│       │       ├── AdminListings.jsx        # Trade listings moderation
│       │       ├── AdminMeters.jsx          # Grid telemetry & manual overrides
│       │       ├── AdminPricing.jsx          # DISCOM rate benchmarks & surge multipliers
│       │       ├── AdminReports.jsx         # Analytical export reports
│       │       ├── AdminDisputes.jsx        # Dispute arbitration panel
│       │       └── AdminCarbon.jsx          # Carbon credit pool tracking
│       ├── auth/                            # Authentication Screens
│       │   ├── Login.jsx                    # Sign in form
│       │   └── Register.jsx                 # Registration (Prosumer/Consumer/Admin)
│       ├── consumer/                        # Consumer Portal Views
│       │   ├── ConsumerDashboard.jsx        # Green energy usage overview
│       │   ├── ConsumerOrders.jsx          # Purchased energy orders
│       │   ├── ConsumerLedger.jsx          # Dedicated buyer purchase history
│       │   ├── ConsumerDisputes.jsx        # Dispute filing interface
│       │   └── ConsumerCarbon.jsx          # Purchased carbon offsets & environmental impact
│       ├── prosumer/                       # Prosumer Portal Views
│       │   ├── ProsumerDashboard.jsx        # Solar generation & earnings overview
│       │   ├── ListingsPage.jsx            # Create & manage surplus energy listings
│       │   ├── ProsumerSmartMeter.jsx      # Live IoT telemetry simulation
│       │   ├── ProsumerCarbon.jsx          # Earned carbon credit manager
│       │   └── ProsumerCertificates.jsx     # Downloadable REC certificates
│       └── shared/                         # Authenticated Common Views
│           ├── Marketplace.jsx              # Real-time P2P energy trading marketplace
│           ├── WalletPage.jsx               # Digital wallet deposits, withdrawals & ledger
│           ├── TransactionsPage.jsx         # General transaction history
│           └── ProfilePage.jsx              # Account settings & KYC management
│   ├── App.jsx                             # Main React Router setup
│   ├── main.jsx                            # Entry point
│   └── index.css                           # Tailwind CSS global styles
├── package.json                            # Frontend dependencies
└── vite.config.js                          # Vite build configuration

server/                                     # Node.js + Express REST API Backend
├── config/                                # Infrastructure Configurations
│   ├── db.js                              # MongoDB Mongoose connection
│   └── seeder.js                           # Initial database seed script
├── controllers/                           # Controller Business Logic
│   ├── adminController.js                  # Admin stats & management logic
│   └── authController.js                   # User auth, JWT issuance & profile updates

```

```

├── certificateController.js # Carbon credit minting & certificate generation
├── disputeController.js    # Trade dispute filing & arbitration logic
├── listingController.js    # Energy listing creation, buying & cancellation
├── matchingController.js   # Proximity & price order matching engine
├── meterController.js      # IoT telemetry & history
├── notificationController.js # In-app notifications
├── pricingController.js    # Tariff rules & surge pricing engine
├── transactionController.js # Financial transaction queries & ledger isolation
├── walletController.js     # Wallet operations & bank payout requests
├── middleware/            # Custom Express Middleware
│   └── auth.js            # JWT validation (`protect`) & Role check (`authorize`)
├── models/                # Mongoose Database Schemas
│   ├── User.js            # User profile & solar panel capacity
│   ├── Listing.js         # P2P energy trade listings
│   ├── Transaction.js     # Escrow transactions & trade logs
│   ├── Wallet.js          # User balances & financial ledger
│   ├── MeterReading.js    # Time-series smart meter readings
│   ├── PricingSetting.js  # Dynamic pricing configuration
│   ├── CarbonCredit.js    # Minted carbon credits
│   ├── Dispute.js         # Trade dispute records
│   └── Notification.js    # User notification queue
├── routes/                # Express REST Routes
│   ├── adminRoutes.js
│   ├── authRoutes.js
│   ├── certificateRoutes.js
│   ├── disputeRoutes.js
│   ├── listingRoutes.js
│   ├── matchingRoutes.js
│   ├── meterRoutes.js
│   ├── notificationRoutes.js
│   ├── pricingRoutes.js
│   ├── transactionRoutes.js
│   └── walletRoutes.js
├── services/              # Service Modules
│   └── meterSimulator.js  # Live IoT meter simulator emitting Socket.IO events
├── utils/                 # Helper Functions
│   └── generateToken.js   # JWT token generator
├── .env.example           # Environment schema template
├── package.json           # Backend dependencies
└── server.js              # Main Express app & WebSocket server

```

3. REST API Specifications (All 11 Modules)

All protected endpoints require a valid JWT token passed in the `Authorization: Bearer <token>` header or HTTP-only cookies.

3.1 Authentication Module (`/api/auth`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/auth/register</code>	POST	Public	Registers a new user (Prosumer/Consumer/Admin) and initializes wallet.	<code>{ name, email, password, role, phone, city, capacityKw, adminCode }</code>	<code>201 Created</code> with User & JWT
<code>/api/auth/login</code>	POST	Public	Authenticates credentials and returns JWT token & user object.	<code>{ email, password }</code>	<code>200 OK</code> with User & JWT
<code>/api/auth/logout</code>	POST	Public	Clears session cookies.	None	<code>200 OK</code> <code>{ message: "Logged out" }</code>
<code>/api/auth/me</code>	GET	Protected	Fetches current user profile data.	None	<code>200 OK</code> User object
<code>/api/auth/profile</code>	PATCH	Protected	Updates user details, solar capacity, or KYC numbers.	<code>{ name, phone, city, capacityKw, aadhaarNumber, panNumber }</code>	<code>200 OK</code> updated User
<code>/api/auth/change-password</code>	PUT	Protected	Updates account password.	<code>{ currentPassword, newPassword }</code>	<code>200 OK</code> <code>{ message: "Password updated" }</code>

3.2 Energy Listings Module (`/api/listings`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/listings</code>	GET	Public	Fetches all active surplus energy listings.	Query params: <code>status</code> , <code>minPrice</code> , <code>maxPrice</code>	<code>200 OK</code> Listing array
<code>/api/listings</code>	POST	Prosumer	Creates a new energy sale listing.	<code>{ energyAmount, pricePerKwh, deliveryTime }</code>	<code>201 Created</code> Listing object
<code>/api/listings/:id/buy</code>	POST	Consumer	Purchases an energy listing via escrow wallet deduction.	URL param: <code>id</code>	<code>200 OK</code> Transaction object
<code>/api/listings/:id</code>	DELETE	Prosumer/Admin	Cancels an unsold listing.	URL param: <code>id</code>	<code>200 OK</code> <code>{ message: "Cancelled" }</code>

3.3 Smart Meter Telemetry Module (`/api/meter`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/meter/readings</code>	GET	Protected	Returns historical smart meter generation/consumption readings.	Query param: <code>range</code> (<code>1h</code> , <code>24h</code> , <code>7d</code>)	<code>200 OK</code> time-series array
<code>/api/meter/live</code>	GET	Protected	Instantaneous snapshot of live kW generation and voltage.	None	<code>200 OK</code> live telemetry object
<code>/api/meter/simulate</code>	POST	Prosumer/Admin	Simulates custom smart meter load pulse.	<code>{ generationKw, consumptionKw, gridVoltage }</code>	<code>200 OK</code> telemetry payload

3.4 Financial Ledger Module (`/api/transactions`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/transactions</code>	GET	Protected	Returns user's transaction ledger (Isolated view for buyer purchases / seller sales).	Query params: <code>type</code> , <code>page</code> , <code>limit</code>	<code>200 OK</code> paginated ledger
<code>/api/transactions/:id</code>	GET	Protected	Retrieves detailed receipt for a transaction.	URL param: <code>id</code>	<code>200 OK</code> detailed Transaction

3.5 Digital Wallet Module (`/api/wallet`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/wallet</code>	GET	Protected	Fetches current balance and linked bank account details.	None	<code>200 OK</code> Wallet object
<code>/api/wallet/deposit</code>	POST	Protected	Deposits virtual demo funds into wallet.	<code>{ amount }</code>	<code>200 OK</code> updated Wallet
<code>/api/wallet/withdraw</code>	POST	Protected	Requests withdrawal to linked bank account.	<code>{ amount, bankDetails }</code>	<code>200 OK</code> withdrawal log

3.6 Dynamic Pricing Module (`/api/pricing`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/pricing/current</code>	GET	Public	Returns DISCOM rate, price ceiling/floor, and surge multiplier.	None	<code>200 OK</code> PricingSetting object
<code>/api/pricing/update</code>	PUT	Admin	Updates system tariff limits and surge rules.	<code>{ discomRate, minPrice, maxPrice, surgeMultiplier }</code>	<code>200 OK</code> updated settings

3.7 Automated P2P Matching Module (`/api/matching`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/matching/recommend</code>	GET	Consumer	Returns listings ranked by distance, price, and seller rating.	Query params: <code>maxPrice</code> , <code>preferredDistance</code>	<code>200 OK</code> ranked listings
<code>/api/matching/auto-trade</code>	POST	Consumer	Automatically executes instant trade matching for best price.	<code>{ maxBudget, requiredKwh }</code>	<code>200 OK</code> executed trade

3.8 Carbon Credit Module (`/api/certificates`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/certificates</code>	GET	Protected	Returns list of earned Green REC certificates and CO_2 offsets.	None	<code>200 OK</code> certificate array
<code>/api/certificates/claim</code>	POST	Prosumer	Claims carbon credits earned from solar generation.	<code>{ creditIds }</code>	<code>200 OK</code> claimed certificate

3.9 Dispute Resolution Module (`/api/disputes`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/disputes</code>	GET	Protected	Lists open trade delivery disputes.	None	<code>200 OK</code> Dispute array
<code>/api/disputes</code>	POST	Protected	Files a trade discrepancy dispute.	<code>{ transactionId, reason, description }</code>	<code>201 Created</code> Dispute
<code>/api/disputes/:id/resolve</code>	PUT	Admin	Arbitrates dispute and releases escrow or refund.	<code>{ resolution, refundAmount }</code>	<code>200 OK</code> resolved Dispute

3.10 In-App Notifications Module (`/api/notifications`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/notifications</code>	GET	Protected	Returns user notifications.	None	<code>200 OK</code> Notification list
<code>/api/notifications/read</code>	PATCH	Protected	Marks notifications as read.	<code>{ notificationIds }</code>	<code>200 OK</code> updated status

3.11 Admin Panel Module (`/api/admin`)

Endpoint	Method	Access	Description	Request Body / Parameters	Response
<code>/api/admin/stats</code>	GET	Admin	Overall grid stats, total energy traded, and active users.	None	<code>200 OK</code> analytics payload
<code>/api/admin/users</code>	GET	Admin	List and search user accounts.	Query params: <code>role</code> , <code>search</code>	<code>200 OK</code> user array
<code>/api/admin/users/:id/status</code>	PATCH	Admin	Activates or suspends user accounts.	<code>{ status }</code>	<code>200 OK</code> updated user

4. MongoDB Database Schemas & Modeling

SolarShare uses 9 Mongoose schemas in MongoDB Atlas:

```
// 1. User Schema (User.js)
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true, select: false },
  role: { type: String, enum: ["prosumer", "consumer", "admin"], required: true },
  phone: String,
  address: { city: String },
  solarPanel: { capacityKw: { type: Number, default: 0 } },
  carbonCreditsTotal: { type: Number, default: 0 },
  aadhaarNumber: String,
  panNumber: String,
  bankDetails: { accountNo: String, bankName: String, ifsc: String }
}, { timestamps: true });

// 2. Listing Schema (Listing.js)
const listingSchema = new mongoose.Schema({
  seller: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  energyAmount: { type: Number, required: true }, // in kWh
  pricePerKwh: { type: Number, required: true }, // in INR
  totalPrice: { type: Number, required: true },
  status: { type: String, enum: ["active", "sold", "cancelled"], default: "active" },
  deliveryTime: Date
}, { timestamps: true });

// 3. Transaction Schema (Transaction.js)
const transactionSchema = new mongoose.Schema({
  listing: { type: mongoose.Schema.Types.ObjectId, ref: "Listing" },
  buyer: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  seller: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  energyAmount: Number,
  pricePerKwh: Number,
  totalAmount: Number,
  status: { type: String, enum: ["completed", "disputed", "refunded"], default: "completed" }
}, { timestamps: true });

// 4. Wallet Schema (Wallet.js)
const walletSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true, unique: true },
  balance: { type: Number, default: 0 },
  ledger: [{
    type: { type: String, enum: ["deposit", "withdrawal", "purchase", "sale", "refund" ] },
    amount: Number,
    description: String,
    createdAt: { type: Date, default: Date.now }
  ]
}, { timestamps: true });
```

(Additional schemas include `MeterReading.js`, `PricingSetting.js`, `CarbonCredit.js`, `Dispute.js`, and `Notification.js`)

5. Technologies & Express Libraries Breakdown

5.1 Backend Technologies & Packages (`server/package.json`)

- `express` (v4.19.2): Core web framework handling route mapping, request parsing, and response middleware.
- `mongoose` (v8.24.1): Object Data Modeling library managing MongoDB Atlas database schemas, validation rules, and queries.
- `jsonwebtoken` (v9.0.2): Generates and verifies stateless JWT tokens for secure authentication.
- `bcryptjs` (v2.4.3): Hashes user passwords with salted key iteration before storing in database.
- `cors` (v2.8.5): Configures Cross-Origin Resource Sharing headers for cross-domain communication between Vercel and Render.
- `dotenv` (v16.4.5): Reads `.env` key-value pairs into `process.env`.
- `cookie-parser` (v1.4.6): Parses cookie headers on incoming requests.
- `socket.io` (v4.7.5): Powers WebSocket real-time telemetry streaming from the smart meter simulator.
- `nodemon` (v3.1.0) (*Dev Dependency*): Auto-restarts server process upon backend code changes.

5.2 Frontend Technologies & Packages (`client/package.json`)

- `react` & `react-dom` (v18.2.0): Component-based UI engine.
 - `react-router-dom` (v6.22.3): Handles single-page application routing and page transitions.
 - `axios` (v1.6.7): HTTP request client configured with request interceptors for token headers.
 - `lucide-react` (v0.344.0): SVG icon library for dashboard icons.
 - `recharts` (v2.12.2): SVG chart generation library for live energy & revenue charts.
 - `socket.io-client` (v4.7.5): Client-side WebSocket listener for live meter updates.
 - `vite` (v5.1.4): Ultra-fast frontend bundler.
 - `tailwindcss` (v3.4.1): Utility-first CSS engine.
-

6. Real-Time Use Cases & Competitive Edge (Why SolarShare is Better)

6.1 Direct Comparison: DISCOM Net-Metering vs SolarShare P2P

Feature	DISCOM Net-Metering	SolarShare P2P Platform	Competitive Superiority
Prosumer Sell Price	₹2.00 – ₹3.00 / kWh	₹4.50 – ₹5.50 / kWh	+60% Higher Income for Prosumers
Consumer Buy Price	₹7.50 – ₹10.00 / kWh	₹5.50 – ₹6.50 / kWh	Up to 30% Electricity Bill Savings
Payment Cycle	Monthly/Quarterly adjustment	Instant Escrow Settlement	Zero waiting time; immediate wallet payout
Green Verification	No green proof	Tokenized Carbon Certificates (REC)	Verifiable environmental offset tracking
Control	Utility monopoly dictates rate	Free Market Demand/Supply	Prosumers choose custom pricing

6.2 Real-Time Key Use Cases

Use Case A: Day-Time Solar Surplus Monetization

- **Scenario:** A prosumer generates 30 kWh on a sunny day but only uses 10 kWh at home.
- **SolarShare Action:** The prosumer lists 20 kWh at ₹5.00/kWh. A nearby retail shop owner purchases it for ₹100.
- **Result:** Prosumer earns ₹100 immediately (instead of ₹40 under DISCOM), and the shop owner saves ₹60 compared to grid tariffs.

Use Case B: Electric Vehicle (EV) Charging Optimization

- **Scenario:** An EV owner needs to charge their car during peak afternoon hours.
- **SolarShare Action:** The automated matching engine finds the cheapest solar listing within a 2 km radius.
- **Result:** The EV owner charges on 100% verified clean solar power at ₹5.50/kWh instead of grid peak tariff (₹9.50/kWh).

7. How to Work on the Project (Setup & Execution Guide)

Step 1: Environment Setup

1. Open terminal and go to the server directory:

```
cd D:\SolarShare\server
```

2. Create `.env` file from `.env.example` :

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/solarshare
JWT_SECRET=your_long_random_jwt_secret_key
CLIENT_URL=http://localhost:5173
ADMIN_REGISTRATION_CODE=solar@admin123
```

Step 2: Start Backend Server

```
cd D:\SolarShare\server
npm install
npm run dev
```

Server starts on `http://localhost:5000` with MongoDB connected and seed data initialized.

Step 3: Start Frontend Client

```
cd D:\SolarShare\client
npm install
npm run dev
```

Client opens on `http://localhost:5173` .

Step 4: Testing & Role Verification

Login using preset demo credentials:

- **Admin:** `admin@solarshare.com` / `password123`
- **Prosumer:** `prosumer1@solarshare.com` / `password123`
- **Consumer:** `consumer1@solarshare.com` / `password123`

8. Live Production Deployments

- **Frontend App:** <https://solar-share-seven.vercel.app>
- **Backend API:** <https://solarshare-05yw.onrender.com>
- **GitHub Repository:** <https://github.com/Samridhi21/SolarShare>